



UNIVERSIDAD
COMPLUTENSE
MADRID

nticmaster

Trabajo Final de Máster

Sistema de recomendación y similitud vectorial para la personalización de catálogo y descubrimiento de productos (Plataforma Steam)

Alumno: **Max Jahaziel Mondragón Espinoza**

Identificación: F798322

Modalidad Semi-presencial

Tutores: Carlos Ortega y Santiago Mota

Máster Data Science, Big Data & Business Analytics
UCM

Septiembre 2026

Índice de Contenido

Introducción	1
Objetivos	2
Objetivo General:	2
Objetivos Específicos:	2
Capítulo 1: Planteamiento del problema y Filtrado Colaborativo	3
Capítulo 2: Recolección y preparación de datos	4
Capítulo 3: Modelado con ALS	7
Modelo 0:	7
Métricas de Evaluación	8
Evaluación Modelo 0	8
Evaluación Azar	8
Evaluación Solo Top 10 siempre	8
Prueba de recomendaciones	10
Tuneo de Hiperparámetros	11
Alpha	11
Factors	11
Regularización	12
Iteraciones	12
Capítulo 4: Inferencia y Artefactos	13
Extracción de artefactos	13
Arquitectura de la solución	15
Capítulo 5: Prueba de Concepto	17
Recomendaciones "Por ID de Usuario"	17
Recomendaciones "Por Similitud de Vectores"	18
Conclusiones	19
Próximos pasos	19
Bibliografía	20

Introducción

El mundo moderno está cada vez más lleno de todo tipo de contenido, si hablamos de entretenimiento, encontramos desde películas, libros, hobbies, series, videojuegos y un sinnúmero de opciones para cada uno. Por otro lado, nuestro tiempo como consumidores es el mismo: 24 horas al día.

De esta premisa derivamos una necesidad que ha existido hace ya muchos años, pero que hoy se ha vuelto cada vez más prioritaria.

Ser selectivo y llegar a los consumidores con productos relevantes, aprovechar la cada vez más limitada ventana de atención y ser capaces de captar su interés.

Este tipo de problema se vuelve especialmente desafiante cuando el volumen de datos crece a gran escala.

El objetivo de este trabajo es diseñar, entrenar y desplegar un sistema de recomendación basado en Filtrado Colaborativo con ALS, capaz de realizar recomendaciones personalizadas a cada usuario tomando para ello los datos de más de 100 millones de reseñas desde la plataforma de videojuegos Steam, e implementarlo como una Prueba de Concepto funcional con inferencia en tiempo real.

Si bien los datos que alimentan este modelo corresponden a una biblioteca de videojuegos, los principios son aplicables a conjuntos de datos comunes donde existan Usuarios e Ítems.

Objetivos

Objetivo General:

Entrenar y desplegar un modelo de motor de recomendaciones mediante filtrado colaborativo, basado en más de 100 millones de reseñas de jugadores en la plataforma Steam, que refleje los gustos de los usuarios y las similitudes entre videojuegos, con el fin de generar recomendaciones relevantes y personalizadas para cada usuario.

Objetivos Específicos:

- Analizar y preparar un gran volumen de datos de forma eficiente, permitiendo su tratamiento con recursos de memoria (RAM) y procesamiento (CPU/GPU) limitados, para obtener de estos datos los registros válidos y útiles para presentar al modelo.
- Seleccionar el algoritmo apropiado para el problema a resolver, definir consideraciones y limitaciones, así como los motivos de su selección.
- Definir un método de evaluación válido que permita cuantificar el rendimiento del modelo, su comparación con el azar o una simple lista de ítems más populares.
- Entrenar una versión cero del modelo, detallar tiempo de entrenamiento, separar datos de entrenamiento y prueba. Realizar la validación del modelo y evaluar su desempeño.
- Realizar el tuneo de hiperparámetros con el fin de identificar los valores que mejor se ajustan a los datos, proporcionando el mejor balance entre desempeño y esfuerzo.
- Generar un modelo de recomendación final, evaluar su desempeño y extraer sus vectores latentes en preparación para su despliegue.
- Elaborar una prueba de concepto donde se hace disponible el modelo entrenado y sus componentes para inferencia (predicción) en tiempo real, seleccionando plataformas y servicios con alta disponibilidad, pero también un enfoque en la optimización de costos.

Capítulo 1: Planteamiento del problema y Filtrado Colaborativo

Steam es la plataforma de distribución de videojuegos para computadoras más grande de los últimos años, con cerca de un 75% del mercado de videojuegos digitales a nivel mundial y un total de 147 millones de usuarios activos mensuales (SQ Magazine, 2026), sumado a los casi 140,000 juegos distintos (SteamDB, 2026) registrados en 2026. Es sin duda un coloso en términos de volumen de información, donde cada usuario puede no solo adquirir videojuegos sino también comentar, interactuar con otros usuarios, compartir contenido y dejar una valoración.

Este último punto tiene un gran valor desde la perspectiva analítica; la capacidad de generar recomendaciones personalizadas partiendo de lo que llamamos "Filtrado Colaborativo".

El filtrado colaborativo es un método de recuperación de información que recomienda elementos a los usuarios en función de cómo otros usuarios con preferencias y comportamientos similares, interactuaron con ese elemento (Murel & Kavlakoglu, 2026).

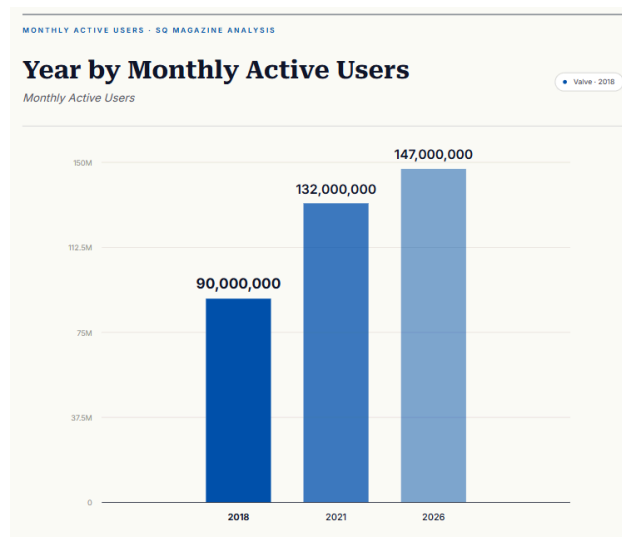


Figura 1. Usuarios activos mensuales (SQ Magazine, 2026)

Aplicado a las reseñas de videojuegos, obtenemos la premisa que, mediante la aplicación de ciencia de datos y modelos de machine learning, es posible realizar recomendaciones personalizadas, siendo para ello necesario computar las interacciones de usuarios y juegos en forma de una matriz, donde podríamos decir que los usuarios son las filas, y los videojuegos son las columnas.

Ahora bien, si intentamos realizar una matriz de esta forma, rápidamente encontraremos no solo un área gigantesca, sino también que la inmensa mayoría de las columnas estarán en cero, dicho de otra forma, es imposible que un usuario haya jugado todos los más de 140,000 juegos (columnas) en la plataforma, siendo optimistas en su vida quizás haya jugado unos 100-200 y esto incluso generoso.

Desde el otro punto de vista, es igualmente imposible que un juego haya sido jugado por todos y cada uno de los más de 147 millones de jugadores (filas), dejando igualmente una mayoría de ceros.

Este problema es justamente lo que denominamos una Matriz Dispersa (Sparse Matrix) y para el cual el algoritmo ALS (Alternating Least Squares) o Mínimos Cuadrados Alternos aborda mediante una técnica de factorización de matrices donde esta matriz dispersa se descompone en dos matrices densas de menor dimensionalidad, una para usuarios y otra para ítems.

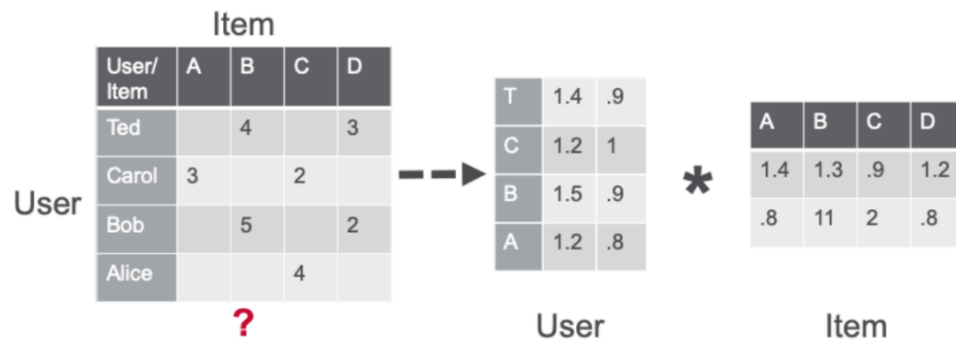


Figura 2. Conversión matriz dispersa en matrices densas. (NVIDIA, 2026)

La matriz sparse R , puede de esta forma aproximarse al producto de las matrices U e I , donde U e I son los **factores**, también llamados vectores latentes.

$$R \approx U \cdot I$$

La forma en que este algoritmo converge es bastante interesante ya que literalmente “Alterna” entre solucionar los factores del ítem que resuelven para los usuarios, y luego soluciona los factores de usuario que resuelven para los ítem; repitiendo en múltiples iteraciones buscando minimizar la diferencia al cuadrado entre lo predicho y lo real.

Capítulo 2: Recolección y preparación de datos

La plataforma Steam proporciona de forma gratuita una API (Interfaz de programación de aplicaciones) que permite consultar directamente las reseñas (y mucha más información) dejadas por usuarios para cada juego o app id mediante un llamado similar a:

https://store.steampowered.com/appreviews/<APP_ID>

Gracias al esfuerzo de la comunidad, existen a nuestra disposición sets de datos previamente extraídos con este método, centralizados en plataformas como kaggle, donde podemos encontrar desde catálogos de juegos, perfiles de usuarios y, por supuesto, reseñas de los jugadores.

Después de una búsqueda se identificaron dos datasets con la información necesaria.

Steam Games Dataset, cortesía de Martín Bustos con licencia MIT.

100 Million+ Steam Reviews, usuario KieranPO'C, también con licencia MIT.

El primer dataset corresponde al catálogo de videojuegos de Steam, con más de 125,000 juegos registrados abarcando 401 MB. Cubre muy bien la necesidad de metadatos, mayormente traducir AppID al nombre del videojuego, así como ayudarnos más adelante a separar juegos de DLC o herramientas.

El segundo set de datos corresponde a un segmento muchísimo mayor de la información, las reseñas individuales dejadas por cada usuario.

Solamente el archivo de reviews tiene un tamaño de 42.49 GB, con más de 110 millones de registros supone un reto tanto a nivel de procesamiento, como del propio entrenamiento del modelo.

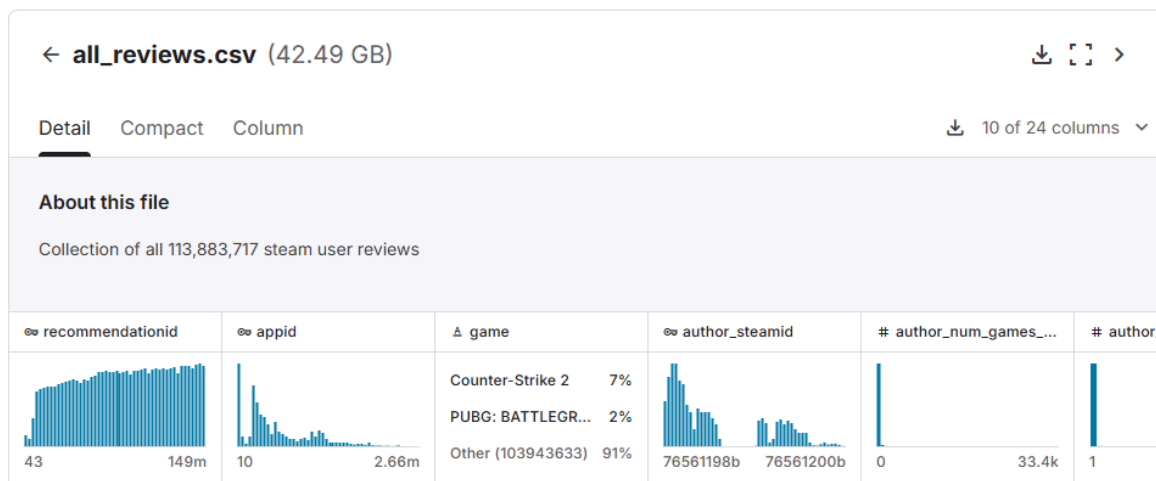


Figura 3. 100 millones de reviews. Kaggle/KieranPO'C

Como primer paso en el tratamiento de grandes volúmenes de información, se decide convertir estos archivos de un formato CSV a un formato más apropiado y utilizado comúnmente en la Ingeniería de Datos, parquet.

La sección de anexos contiene la referencia al Notebook de Jupyter con el código requerido para esta conversión.

Como resultado final se obtienen dos archivos, games.parquet con apenas 110MB de espacio, y reviews.parquet con 16.5 GB.

En adelante se utilizan estos archivos pre-procesados por su facilidad de utilización con librerías como Spark o Pandas, así como su uso óptimo de datos al momento de explorar y consultar con un motor de Base de Datos (BigQuery, comentado más adelante).

El notebook *01_exploracion_etl* define todos los pasos de limpieza y preparación de datos a detalle, pero se mencionan a continuación algunas decisiones y consideraciones importantes.

El dataset contiene 37,918,537 usuarios únicos, así como 105,893 juegos. Con un balance de clases bastante inclinado a valoraciones positivas, 85.6% con apenas un 14.4% de reseñas negativas.

Hay dos gráficos que agrupan dos análisis importantes, total de reviews por usuario, y total de reviews por juego.

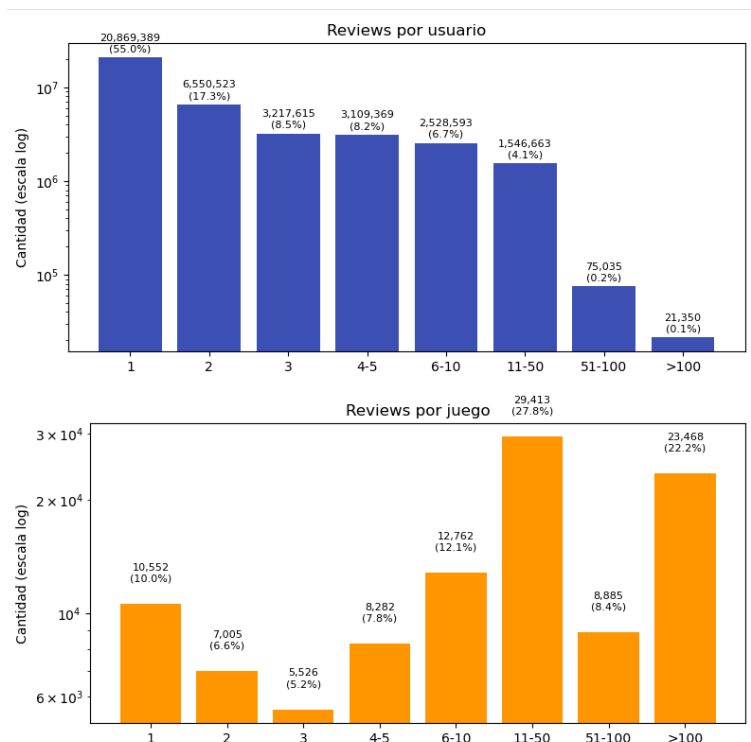


Figura 4. Conteos por usuario y por juego.

Por su naturaleza, el algoritmo ALS no tiene nada que aprender de un usuario que interactuó con solamente 1 ítem. Es decir no puede crear relaciones ya que simplemente no existe otro ítem asociado a este usuario. Lo mismo aplica para un juego con solamente 1 reseña.

Por este motivo se toma la decisión de considerar solamente los usuarios con 2 o más reseñas, así como juegos con 5 o más reseñas. Este último ya que tampoco queremos incluir juegos extremadamente *niche*.

Otro paso necesario es el encoding a enteros consecutivos. En lugar de guardar los ID puros (ejemplo: 76561198324251650) los cuales se convierten a índices de 0 hasta n tanto para jugadores (users) como para juegos (items).

Esto no solo es una optimización en el uso de memoria, sino también un requisito obligatorio para el modelo ALS a utilizar más adelante.

Capítulo 3: Modelado con ALS

Actualmente se cuenta con 3 archivos pre-procesados para el entrenamiento:

- steam_reviews_clean.parquet: Archivo preparado, sin duplicados y con 92,958,418 registros, 17,049,073 usuarios y 78,376 juegos.
- user_map.parquet: Map de user_id con user_idx, permite recuperar los ID originales de usuario, a partir de los enteros consecutivos generados.
- item_map.parquet: Map de item_id con item_idx, permite recuperar los ID originales de cada videojuego, a partir de los enteros consecutivos.

La librería a utilizar para el entrenamiento del modelo es el paquete **implicit**, una librería de Filtrado Colaborativo escrita en Python por Ben Frederickson y compartida con licencia MIT.

Modelo 0:

Se ejecuta una primera fase de entrenamiento con valores de hiperparámetros genéricos (se explica en la sección de Tuneo de hiperparámetros), es decir un número sin fundamento real. Por ejemplo al seleccionar el número 128 para la cantidad de factores, se da un amplio margen al modelo para capturar interacciones complejas. Podríamos verlo como cuantas dimensiones o coordenadas son necesarias para posicionar un vector en la dirección correcta.

```
# Libreria implicit, Licencia MIT Copyright (c) 2016 Ben Frederickson
# https://github.com/benfred/implicit.git
from implicit.als import AlternatingLeastSquares
from threadpoolctl import threadpool_limits

# Limita BLAS a 1 hilo para evitar conflictos
threadpool_limits(1, "blas")

model = AlternatingLeastSquares(
    factors=128,
    regularization=0.05,
    iterations=20,
    num_threads=0,      # 0 = todos los cores
    random_state=42,
)

t0 = time.time()
model.fit(train_matrix, show_progress=True)
print(f"\nEntrenamiento completo en {(time.time()-t0)/60:.1f} min")
```

Figura 5. Entrenamiento de Modelo 0

El resultado del modelo entrenado se guarda en la variable *model*. Con esto podemos continuar con la fase de evaluación, así como algunas pruebas puntuales de recomendación.

Métricas de Evaluación

Para fines de evaluación del modelo, se decide utilizar la técnica de "Leave one out", o dejar uno por fuera. Ya que el dataset preparado contiene usuarios con 2 o más reviews, podemos separar los segmentos de Train y Test dejando una única review en test (la última) y usando todas las demás en el entrenamiento, esto hará que el modelo intente predecir ese único juego que quedó por fuera.

Mediremos entonces la métrica llamada HitRate@K, donde K corresponde al top de resultados que tomaremos (Por ejemplo recomendar top 10 juegos posibles). Como métrica de evaluación, es bastante estricta ya que en términos generales, el sistema intentará predecir un juego que quedó por fuera, de entre los más de 120,000 disponibles para cada usuario. Esta métrica es análoga a la Precisión evaluando 1 ítem, que justamente es el caso a validar en el ejercicio.

La siguiente métrica a evaluar es, del listado de resultados generados K, en qué posición quedó la recomendación acertada. Por ejemplo si recomendamos 10 ítems pero el ítem que interesa al usuario quedó apenas en la posición 10 y no en los primeros lugares, esto es una señal de problemas en el ranking. Esta métrica es llamada MRR@K o Rango Recíproco Medio.

Se seleccionan entonces dos métricas, HitRate@K y MRR@K para nuestra evaluación.

Evaluación Modelo 0

Hit Rate@10 (precision@10 sobre 1 ítem real): 0.1060

MRR@10 Global: 0.0475, MMR@10 Condicional (para los aciertos): 0.4481

Evaluación Azar

Probabilidad recomendando al azar: $10 / 78376 = 0.0001276$

Probabilidad modelo 0: 0.1060

Comparación: $0.1060 / 0.0001276 = 830.72$

El modelo 0 tiene una tasa de aciertos (HitRate@10) 830 veces mejor que el azar.

Evaluación Solo Top 10 siempre

Una comparación interesante es tomar solo el top n ítems más populares y recomendar únicamente esos a todos los usuarios. Por supuesto no tiene diversidad y descarta el 99.9% de los otros juegos, pero es importante evaluar si nuestro modelo supera o no un simple top.

Popularidad – Hit Rate@10: 0.1190

Como tal, una recomendación simple del Top 10 es ligeramente superior al modelo 0, Con un 11.9% vs 10.6% de acierto respectivamente.

Esto es en cierta medida esperado ya que, si tomamos la gráfica generada en la sección de exploración, podemos observar un fuerte sesgo hacia los juegos más populares, donde el top 10 tiene, en promedio, 2000 veces más reviews que los juegos fuera del top 10.

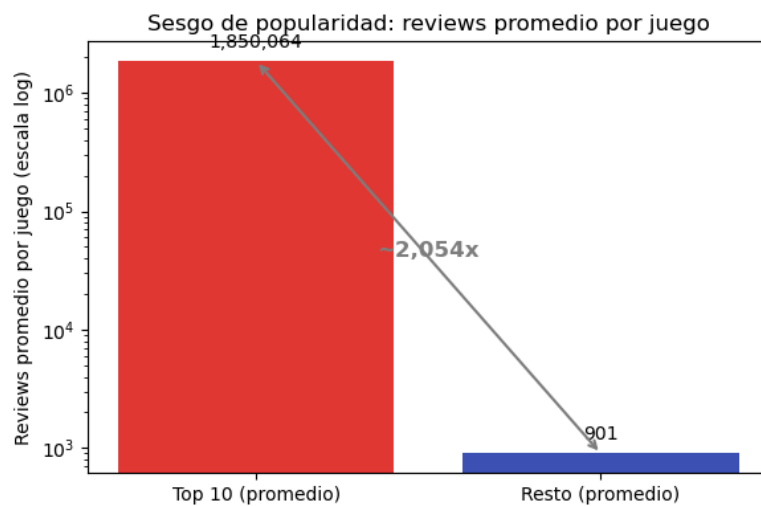


Figura 6. Sesgo de popularidad Top 10 vs los restantes.

Este es un punto importante a evaluar, no solo queremos que el modelo presente mejores recomendaciones que el azar o un simple top 10, sino que se espera también diversidad en las recomendaciones. Este punto se extiende en la sección de próximos pasos.

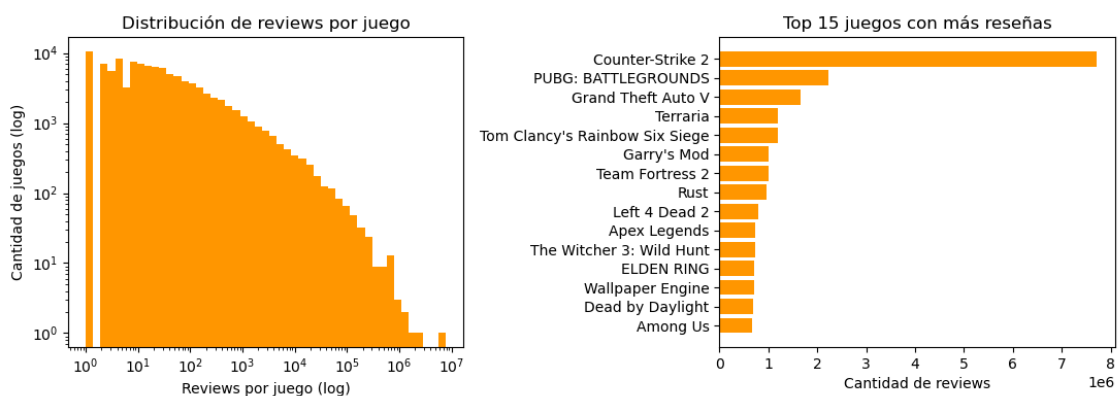


Figura 7. Alta concentración de reviews en juegos Top.

Prueba de recomendaciones

Una evaluación empírica para un usuario con dos reviews (propio) genera los siguientes resultados:

score	AppID	Name	Acertada Si/No
0.3609	1113560	NieR Replicant™ ver.1.22474487139...	Si
0.3081	578080	PUBG: BATTLEGROUNDS	No
0.2568	814380	Sekiro™: Shadows Die Twice - GOTY Edition	Si
0.2388	678960	CODE VEIN	Si
0.2351	637650	FINAL FANTASY XV WINDOWS EDITION	Si
0.2292	440	Team Fortress 2	No
0.2125	431960	Wallpaper Engine	Si
0.1970	39210	FINAL FANTASY XIV Online	Si
0.1833	447530	VA-11 Hall-A: Cyberpunk Bartender Action	No
0.1777	460790	Bayonetta	Si

En este resultado preliminar se observa algunos de los ítem sumamente populares que se “cuelan” en la lista. Más adelante cuantificamos esto y se tomará la decisión para su tratamiento ya que su popularidad no es un error, sino que genuinamente los usuarios comparten gustos por ese juego así que, dependiendo de la concentración de juegos populares, podríamos aceptar o aplicar una penalización menor.

Por otro lado, mediante similitud de vectores, se obtiene una lista de juegos “similares” tomando el appid 489830 que corresponde a Skyrim como ejemplo. En este aspecto casi todos los ítems son, en efecto, similares. En ningún momento el modelo ha tenido acceso a nombres o géneros del videojuego. Únicamente ha podido ver los IDs numéricos, lo cual es un buen resultado inicial.

score	appid	game	Similar Si/No
1	489830	The Elder Scrolls V: Skyrim Special Edition	Si
0.381	1746860	The Elder Scrolls V: Skyrim Anniversary Upgrade	Si
0.3398	377160	Fallout 4	Si
0.3236	22330	The Elder Scrolls IV: Oblivion	Si
0.3232	900883	The Elder Scrolls IV: Oblivion® Game of the Year Edition Deluxe	Si
0.3004	976620	Enderal: Forgotten Stories (Special Edition)	Si
0.2966	306130	The Elder Scrolls Online	Si
0.2763	204360	Castle Crashers	No?
0.2517	22320	The Elder Scrolls III: Morrowind	Si
0.2292	365720	Skyrim Script Extender (SKSE)	Si

Tuneo de Hiperparámetros

En la sección 7 del notebook 02_modelado_als se detallan todos los hiperparámetros evaluados, así como sus pruebas. En general se parte de un valor y se realizan múltiples pruebas y validaciones para encontrar un punto óptimo para cada uno.

Para agilizar la exploración de hiperparámetros, las pruebas se ejecutan sobre una muestra reducida del dataset (a diferencia del Modelo 0, entrenado con el conjunto completo). Adicionalmente, el ajuste de hiperparámetros se realiza de forma secuencial e individual con escalas de valores definidas.

Alpha

Con valor por defecto de 40 en la librería, este hiperparámetro representa cuánto se confía en una interacción positiva observada por encima del valor base 1, donde ese valor 1 es igual a no observado. (Hu, Koren & Volinsky., 2008).

$$c_{ui} = 1 + \alpha r_{ui}$$

```
alpha= 5 -> Hit Rate@10: 0.0800 | MRR@10: 0.0386 (n=20,000)
alpha= 10 -> Hit Rate@10: 0.0894 | MRR@10: 0.0427 (n=20,000)
alpha= 20 -> Hit Rate@10: 0.0986 | MRR@10: 0.0446 (n=20,000)
alpha= 30 -> Hit Rate@10: 0.1038 | MRR@10: 0.0461 (n=20,000)
alpha= 32 -> Hit Rate@10: 0.1042 | MRR@10: 0.0464 (n=20,000)
alpha= 35 -> Hit Rate@10: 0.1023 | MRR@10: 0.0465 (n=20,000)
alpha= 40 -> Hit Rate@10: 0.1027 | MRR@10: 0.0463 (n=20,000)
alpha= 50 -> Hit Rate@10: 0.1021 | MRR@10: 0.0462 (n=20,000)
alpha= 60 -> Hit Rate@10: 0.1021 | MRR@10: 0.0459 (n=20,000)
alpha= 80 -> Hit Rate@10: 0.0999 | MRR@10: 0.0450 (n=20,000)
```

Poca variación entre 20 y 60, donde el valor 32 genera el mayor Hit Rate sin mayor sacrificio del MRR.

Factors

Este parámetro representa la dimensión del espacio latente. Una comparación sería cuantas coordenadas necesitamos para indicar la dirección del vector en el espacio. Más factores da lugar a más capacidad para captar matices complejos, pero puede dar lugar a sobreajuste. Menos factores por otro lado, podría simplificar demasiado el modelo y volverlo incapaz de señalar la dirección correcta.

```
factors= 8 -> Hit Rate@10: 0.1212 | MRR@10: 0.0502 (n=20,000)
factors= 12 -> Hit Rate@10: 0.1313 | MRR@10: 0.0500 (n=20,000)
factors= 16 -> Hit Rate@10: 0.1288 | MRR@10: 0.0530 (n=20,000)
factors= 24 -> Hit Rate@10: 0.1303 | MRR@10: 0.0529 (n=20,000)
factors= 32 -> Hit Rate@10: 0.1293 | MRR@10: 0.0533 (n=20,000)
factors= 48 -> Hit Rate@10: 0.1244 | MRR@10: 0.0501 (n=20,000)
factors= 64 -> Hit Rate@10: 0.1172 | MRR@10: 0.0496 (n=20,000)
factors= 128 -> Hit Rate@10: 0.1042 | MRR@10: 0.0464 (n=20,000)
factors= 256 -> Hit Rate@10: 0.0839 | MRR@10: 0.0405 (n=20,000)
```

Tuneo Fino

```
factors= 12 -> Hit Rate@10: 0.1289 | MRR@10: 0.0496 (n=100,000)
factors= 16 -> Hit Rate@10: 0.1274 | MRR@10: 0.0513 (n=100,000)
factors= 20 -> Hit Rate@10: 0.1253 | MRR@10: 0.0487 (n=100,000)
factors= 24 -> Hit Rate@10: 0.1277 | MRR@10: 0.0519 (n=100,000)
```

La cantidad de factores fué el hiperparámetro que más impacto tuvo en las métricas de evaluación. Contrarió a lo que se esperaría de un dataset tan grande, más coordenadas en el espacio vectorial empeoraba los resultados.

La explicación es bastante simple, con 256 factores (o 128 en el modelo 0) se presentaba el clásico problema de Overfitting, o Sobreajuste. El modelo memorizaba demasiado el set de datos en training, y al tratar evaluar resultados desconocidos (test) no era capaz de generalizar y sus resultados empeoraron.

Menos coordenadas en un espacio vectorial no solo genera mejores resultados, sino que es mucho más fácil de trabajar ya que pasa de un entrenamiento (con una fracción de los 100 millones de registros) de más de 5 horas a 10 minutos.

Cabe también mencionar que por primera vez, el modelo supera en términos de Hit Rate a una presentación simple del Top 10 de juegos.

Regularización

Aplica penalización L2 sobre la magnitud de los vectores, evitando que se ajusten demasiado a poca evidencia, previniendo overfitting.

```
reg= 0.01 -> Hit Rate@10: 0.1254 | MRR@10: 0.0517 (n=100,000)
reg= 0.05 -> Hit Rate@10: 0.1277 | MRR@10: 0.0519 (n=100,000)
reg= 0.1 -> Hit Rate@10: 0.1283 | MRR@10: 0.0520 (n=100,000)
reg= 0.2 -> Hit Rate@10: 0.1272 | MRR@10: 0.0519 (n=100,000)
```

No supuso un impacto mayor, por lo que se mantiene el valor recomendado 0.1.

Iteraciones

Cuántas repeticiones de optimización se realizan para que el algoritmo ALS alterne entre Items y Usuarios.

```
iterations= 15 -> Hit Rate@10: 0.1283 | MRR@10: 0.0530 (n=20,000)
iterations= 30 -> Hit Rate@10: 0.1367 | MRR@10: 0.0568 (n=20,000)
iterations= 50 -> Hit Rate@10: 0.1402 | MRR@10: 0.0575 (n=20,000)
iterations= 80 -> Hit Rate@10: 0.1436 | MRR@10: 0.0592 (n=20,000)
iterations= 100 -> Hit Rate@10: 0.1440 | MRR@10: 0.0592 (n=20,000)
iterations= 150 -> Hit Rate@10: 0.1434 | MRR@10: 0.0599 (n=20,000)
iterations= 200 -> Hit Rate@10: 0.1442 | MRR@10: 0.0598 (n=20,000)
```

Se observa una mejora gradual alineada al número de iteraciones. Esto indica que ALS aún no había convergido con las pocas iteraciones iniciales.

Si bien los valores superiores aumentan los parámetros, la ganancia es mínima por lo que se toma 100 iteraciones como valor final.

Capítulo 4: Inferencia y Artefactos

Uno de los objetivos clave del modelo entrenado es la realización de predicciones, esto puede hacerse de dos formas, por lotes o en tiempo real.

Las predicciones por lotes (Batch) son útiles para almacenar los resultados para su utilización posterior, por ejemplo en una campaña de marketing por medio de Email, donde podemos calendarizar el envío de ofertas con una lista de juegos que podrían ser de interés para el usuario.

Las predicciones en tiempo real (Online) por otro lado suponen un mayor desafío ya que deben entregarse en cuestión de milisegundos. Un ejemplo de esto es un sitio web donde el sistema debe responder las peticiones con una baja latencia.

Este segundo punto es una de las razones más importantes por las que el algoritmo ALS fue el seleccionado para esta tarea.

Una particularidad de ALS es que, como vimos anteriormente, separa la enorme matriz sparse en dos matrices densas de menor dimensionalidad. Al haber seleccionado 24 factores, justamente creamos dos matrices sencillas, una de jugadores de 17 millones x 24, y una de juegos de 78 mil x 24.

Estas matrices abarcan 1.6 GB y 6 MB respectivamente. Como punto de comparación el Modelo 0 abarca 8.5 GB y debe levantarse en memoria para poder utilizarse (similar a otros modelos, como una red neuronal por ejemplo).

Por otro lado las matrices independientes pueden fácilmente consultarse con un lookup (búsqueda directa por índice), de forma que no necesitamos escanear toda la tabla para generar una predicción, simplemente tomamos los factores del usuario o ítem en cuestión, independiente del resto.

Extracción de artefactos

El notebook *99_extrae_artefactos* contiene el código utilizado para exportar los artefactos necesarios para realizar predicciones. De él se tenemos 4 archivos:

1. *item_map.parquet*: Mapeo simple de Item_Id con Item_Idx
2. *user_map.parquet*: Mapeo simple de User_Id con User_Idx
3. *item_factors.npz*: Matriz de ítems con sus 24 factores
4. *user_factors.parquet*: Matriz de usuarios con sus 24 factores

Los dos primeros son los mapeos simples que se generaron en el Capítulo 2 (Notebook 01) pero que sean necesarios para darle sentido a las predicciones.

Los otros dos archivos corresponden a las matrices individuales. El primero (*item_factors*) está en formato npz ya que apenas abarca 6 MB, por lo que será cargado en memoria directamente

El segundo archivo (*user_factors*) se almacena como parquet, ya que al ser más pesado (1.6 GB) será consultado desde una base de datos indexada en su lugar.

working-area-max / Datasets / model_artifacts / Tables / user_factors

☆

user_factors

🔍 Query

💬 Chat

Open in ▾

👤 Share

SchemaDetailsPreviewInsightsLineageData Profile

Row	user_id	user_factors
1	76561197971583456	-0.072633162140846252
		0.16914880275726318
		-0.22947530448436737
		0.0453047938644886
		0.03539077565073967
		-0.37687820196151733
		-0.657996654510498
		-0.17070823907852173
		-0.454069048166275
		0.4084530770778656
		0.69865608215332031
		0.93449509143829346
		0.16637766361236572
		0.15888576209545135
		-0.14868353307247162
		0.54402512311935425
		-0.12516972422599792
		-0.2014591246843338
		-0.37225547432899475
		0.59670686721801758
		0.0806037113070488
		0.771319568157196
		1.2133302688598633
		0.15760765969753265
2	76561197971763154	-0.0062168142758309841
		-0.0043293870985507965
		-0.0041612070053815842

working-area-max / Datasets / model_artifacts / Tables / item_factors

☆

item_factors

🔍 Query

💬 Chat

Open in ▾

👤 Share

SchemaDetailsPreviewInsightsLineageData Profile

Row	item_id	item_factors
1	50	0.060082051903009415
		-0.01267271488904953
		0.12972109019756317
		0.072936400771141052
		-0.065933406352996826
		0.011058853007853031
		0.16515116393566132
		0.114199198782444
		0.022494424134492874
		0.080686204135417938
		0.059908363968133926
		-0.0665249302983284
		0.058990579098463058
		0.12145023792982101
		-0.03277355432510376
		0.1320948600769043
		-0.045312892645597458
		0.01117636077105999
		0.094561845064163208
		-0.039636246860027313
		0.019962579011917114
		0.10170770436525345
		-0.0052214730530977249
		-0.084926627576351166
2	280	0.034027159214019775
		0.0060321367345750332
		0.091981880366802216

Figura 8. Muestra de los 24 factores generados para usuarios (Izquierda), Items (derecha)

La figura anterior muestra un pequeño ejemplo del contenido de la matriz `user_factors` y la matriz de `item_factors`. Como es de esperar, cada `user_id/item_id` tiene un listado de 24 factores (llámese vector) que almacena las interacciones identificadas del ítem, o del usuario (preferencias).

Utilizando estos artefactos, una predicción no es más que un producto punto entre el vector del usuario y el vector del juego. Básicamente multiplicar 24 números por otros 24 números y sumar. Lo que lo vuelve un cálculo sumamente sencillo capaz de resolverse en microsegundos.

En el caso de un top de recomendaciones, simplemente se multiplica el vector del usuario contra los vectores en `item_factors` (cargado en memoria) y se ordena el resultado.

Arquitectura de la solución

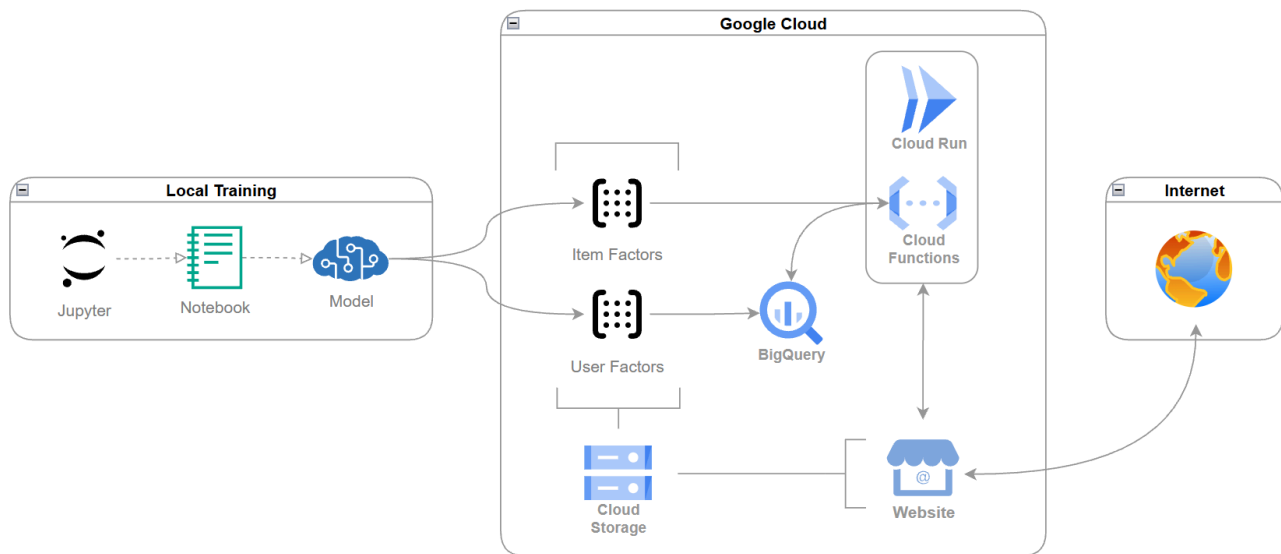


Figura 9. Arquitectura de solución para inferencia en tiempo real

El diagrama muestra la arquitectura seleccionada, incluyendo los diferentes servicios a utilizar, así como sus interacciones.

Se selecciona la plataforma de Google Cloud para los servicios y la prueba de concepto. Entre los motivos que influyen en la decisión está la alta disponibilidad de los servicios, integración nativa entre componentes, rapidez y simplicidad en el despliegue; así como la optimización de costos cuando utilizamos servicios con escalamiento bajo demanda.

La sección izquierda contiene los elementos básicos del entrenamiento, no se requiere hardware especializado como tarjetas gráficas con CUDA o similar. El entrenamiento es perfectamente realizable bajo CPU solamente.

Para fines del proyecto, se entrenó el modelo utilizando un procesador AMD Ryzen 7 9850x3D, si bien es uno de los procesadores comerciales más potentes a la fecha, el tiempo total de entrenamiento con 100 iteraciones y 24 factores fué de poco más de 1 hora, por lo que un procesador de menos capacidad no supone un problema grave.

El siguiente paso para el despliegue en la nube es copiar los artefactos generados en un directorio (bucket) que será accesible solamente dentro de la solución, sin acceso público.

Para ello se selecciona Google Cloud Storage, donde también cargamos otro componente necesario para el sitio web de prueba, la página web estática index.html en un bucket diferente, con permisos de lectura desde fuera de la nube, es decir acceso público.

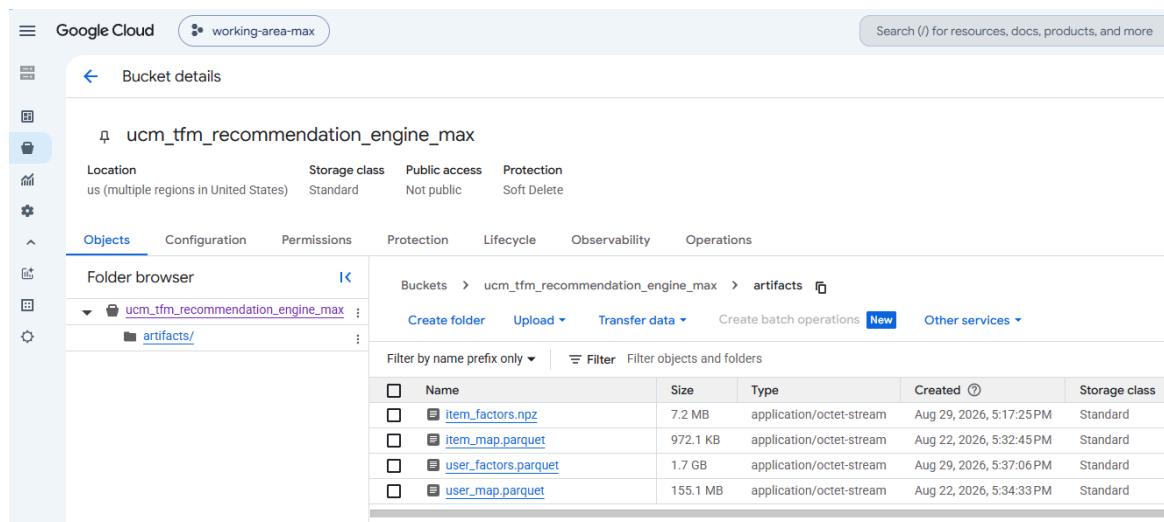


Figura 10. Archivos de artefactos cargados en Cloud Storage

Ahora se prepara un contenedor docker que servirá como intermediario entre el sitio web y los artefactos del modelo entrenado, para responder las peticiones en tiempo real mediante llamados a la API.

El servicio seleccionado es Cloud Run, con su componente de Cloud Functions.

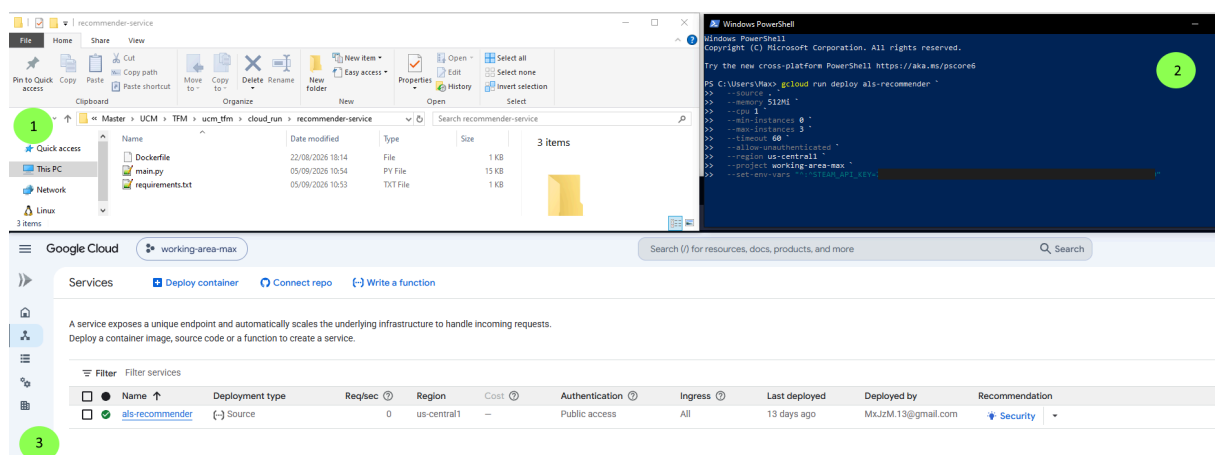


Figura 11. Preparación de archivos (1), empaquetado y carga (2) y contenedor listo (3)

El archivo main.py no es más que una API escrita en python con la librería FastAPI. Un código bastante simple que responde peticiones como por ejemplo Top n recomendaciones para el usuario, o Top n recomendaciones para un perfil, mediante la lectura e interpretación de los archivos de factores (user/item) previamente preparados. Disponible en el repositorio de GitHub.

Este archivo se empaqueta junto a la lista de requerimientos y definiciones dockerfile y se despliega en Cloud Run, como se observa en la figura 11.

Todos los archivos mencionados están disponibles en el repositorio de GitHub, en la sección de anexos.

Capítulo 5: Prueba de Concepto

Una vez desplegada la solución, tenemos en Cloud Storage los factores de usuario y de ítems, las tablas lookup de mapping entre ID e Índices, y la página web estática.

El contenedor docker con la API se encuentra activo en Cloud Run. Y BigQuery permite acceder a alguna información de metadatos extra, como imágenes o URLs.

Se habilita la siguiente dirección web para acceder a la PoC:

<https://storage.googleapis.com/ucm-tfm-recommender-frontend/index.html>

La cual presenta 3 Tabs para interactuar, que se detallan a continuación.

Recomendaciones “Por ID de Usuario”

Esta sección permite realizar recomendaciones por usuario, donde el sistema buscará en su matriz de usuarios el ID especificado, y en caso de existir presentará recomendaciones personalizadas a ese usuario único.

El sitio web desarrollado para esta prueba de concepto permite simular en tiempo real las interacciones de una tienda online , generando recomendaciones en apenas una fracción de segundo, mostrando imágenes y los enlaces a los juegos recomendados.

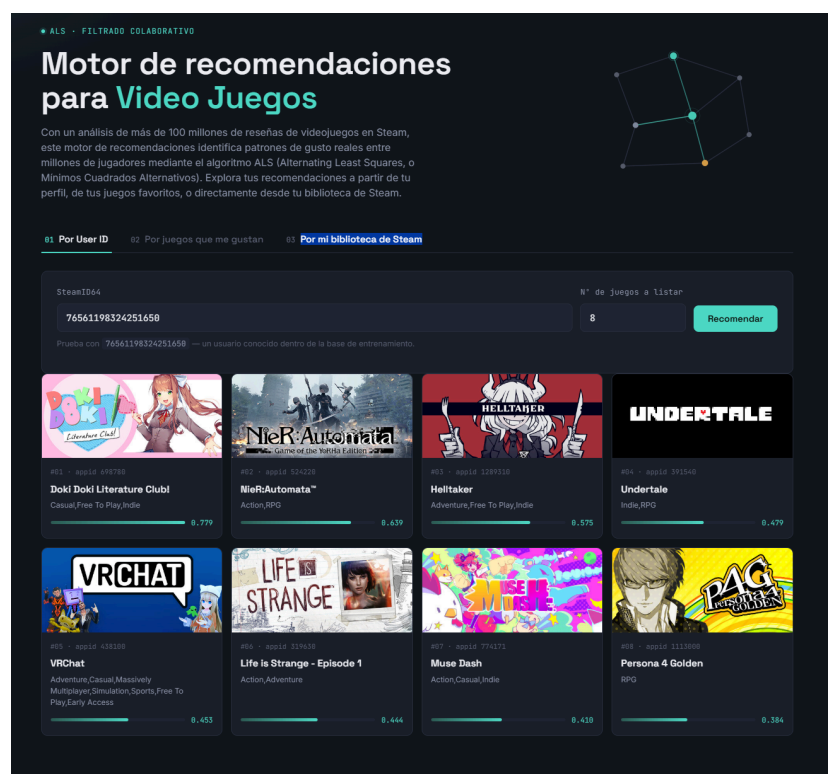


Figura 12. Recomendaciones por ID de usuario

Recomendaciones “Por Similitud de Vectores”

El segundo y tercer Tab, “Por Juegos que me gustan” y “Por mi biblioteca de Steam”, permiten simular gustos de un usuario mediante un producto punto de los factores correspondientes a los juegos seleccionados, promediando los vectores de estos juegos y ordenando los resultados del catálogo resultante.

La similitud vectorial es sumamente útil en escenarios de “arranque en frío”, es decir situaciones donde un usuario es nuevo (desconocido para el modelo), o con información nueva (ahora le gustan uno o más juegos adicionales), y puede recibir recomendaciones personalizadas basadas en un mínimo de información requerida, por ejemplo la lista de juegos que le gustan actualmente.

En la imagen siguiente se seleccionan dos juegos: The Witcher 3 y Elden Ring. Ambos bastante populares pero diferentes, donde el sistema promedia sus vectores y logra recomendar juegos similares, como ser Cyberpunk (mismos creadores de The Witcher), Sekiro (alta dificultad, como Elden Ring), o Skyrim (RPG popular con ciertas similitudes a ambos).

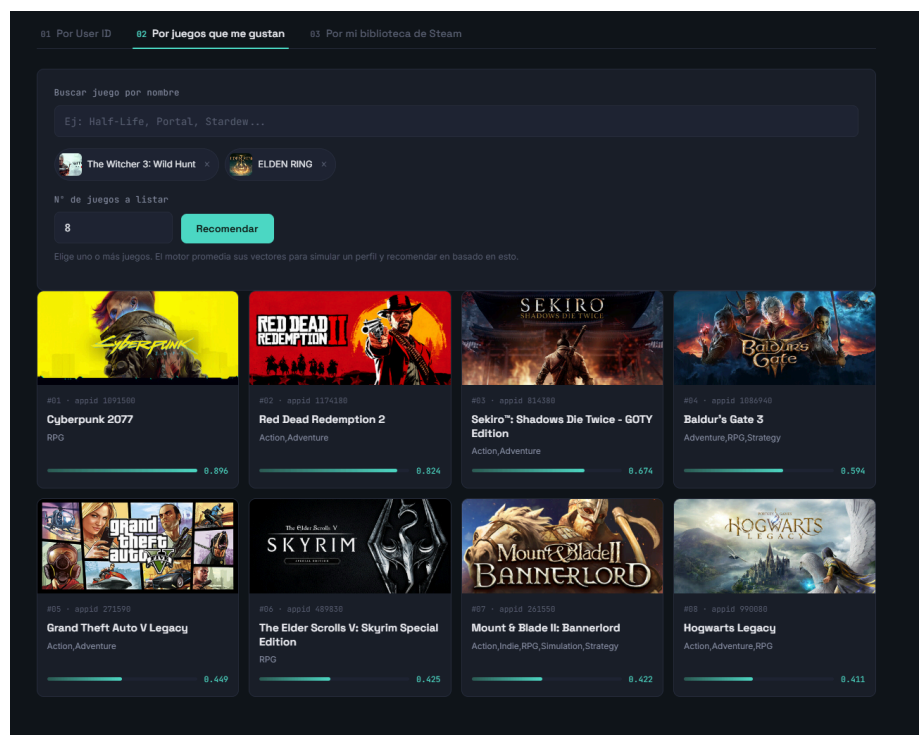


Figura 13. Recomendaciones por similitud vectorial

Un punto importante a mencionar es que el modelo en ningún momento interactuó con el nombre del juego, desarrollador, etiquetas de género o similares. Únicamente tuvo visibilidad de los IDs de juego y jugador, por lo que una recomendación de un juego similar es puramente similitud en el espacio vectorial, derivado de los gustos de millones de usuarios.

Conclusiones

Este trabajo demostró que es posible construir un motor de recomendaciones funcional a partir de un volumen de datos considerable (+100 millones de reseñas) utilizando técnicas de Filtrado Colaborativo, específicamente ALS, incluso sin necesidad de hardware especializado (GPU) para el entrenamiento.

El modelo final, tras el proceso de ajuste de hiperparámetros, logró superar tanto la recomendación aleatoria (1128 veces mejor: 0.144/0.0001276) como una simple lista de ítems populares (+21% HitRate@10 de 0.144 frente a 0.119), validando que el sistema efectivamente capta patrones de preferencia individuales y no sólo reproduce la popularidad general. Un hallazgo relevante fue que reducir la cantidad de factores latentes (de 128 a 24) mejoró el desempeño y redujo drásticamente el tiempo de entrenamiento, evidenciando que el modelo inicial sufría de sobreajuste.

A nivel de arquitectura, la descomposición de la matriz dispersa en dos matrices densas de usuarios e ítems permitió separar el entrenamiento (offline, computacionalmente costoso) de la inferencia (online, un simple producto punto), habilitando recomendaciones en tiempo real con una arquitectura serverless (Cloud Run, Cloud Storage, BigQuery) de bajo costo operativo. La Prueba de Concepto validó este enfoque en tres escenarios: recomendación por usuario existente, y recomendación por similitud vectorial para casos de arranque en frío.

Próximos pasos

- Evaluar la diversidad de las recomendaciones, complementando las métricas de precisión (HitRate, MRR) con métricas que penalicen el sesgo hacia ítems populares, identificado durante la exploración de datos.
- Incorporar señales adicionales de interacción (por ejemplo, horas jugadas) para enriquecer el peso de confianza (alpha) más allá de una reseña binaria positiva/negativa.
- Diseñar un esquema de reentrenamiento periódico y monitoreo de deriva (drift) del modelo en producción, dado que las preferencias y el catálogo de Steam cambian constantemente.
- Considerar autoescalado en respuesta a alta demanda del sitio, así como servicios de baja latencia (Datastore/MongoDB en lugar de BigQuery por ejemplo) para tiempos de respuesta aún más rápidos.

Bibliografía

Hu, Y., Koren, Y., & Volinsky, C. (2008). Collaborative filtering for implicit feedback datasets. 2008 Eighth IEEE International Conference on Data Mining, 263-272.
<https://yifanhu.net/PUB/cf.pdf>

Murel, J., & Kavlakoglu, E. (2026). ¿Qué es el filtrado colaborativo? IBM.
<https://www.ibm.com/mx-es/think/topics/collaborative-filtering>

NVIDIA. (2026). What is a recommendation system?
<https://www.nvidia.com/en-us/glossary/recommendation-system/>

SQ Magazine. (2026, junio). Steam statistics 2026: Users, records, marketshare.
<https://sqmagazine.co.uk/steam-statistics/>

SteamDB. (2026). Steam game release summary by year.
<https://steamdb.info/stats/releases/>

Anexos

Repositorio del proyecto en GitHub:

https://github.com/max-mondragon/ucm_tfm

Carpeta en Drive con archivos adicionales:

<https://drive.google.com/drive/folders/1XKHzyk4DphhU-r7INg6CxGPfTUCHuTnc?usp=sharing>

Sitio web para inferencia (predicciones):

<https://storage.googleapis.com/ucm-tfm-recommender-frontend/index.html>

Costo total de la prueba de concepto en GCP: ~1.57\$

